

System Flow (code reference)

Created	@November 7, 2025 10:52 PM
Tags	

Overall flow

COMPLETE CONSOL APPLICATION FLOW ANALYSIS

DATABASE ARCHITECTURE

Database: PostgreSQL named "Consol" (line 7, db.js)

Tables:

- users (id, username, profile_picture_url, created_at)
- notes (id, user_id, title, content, word_count, created_at)
- sessions (id, user_id, note_id, similarity, stars, word_count, duration_secs, wpm, created_at)

APPLICATION STARTUP FLOW

1. INITIAL ENTRY POINT

File: page.js (Landing Page)

Lines 1-22: Landing page component

- Line 10: setUser(null) - Clears any existing user context
- Line 15: Welcome message and navigation to users page
- Line 18: Link to users page for user selection

2. USER CONTEXT MANAGEMENT

File: UserContext.js

Lines 9-16: UserProvider component with localStorage persistence

- Line 12: localStorage.getItem('consol_user') - Loads saved user on app start
- Line 18-21: setUser function saves to localStorage for persistence

3. ROOT LAYOUT & NAVIGATION

File: layout.js

Lines 23-29: Root layout with UserProvider and Navbar

- Line 25: <UserProvider> wraps entire app for user state management
- Line 26: <Navbar /> provides navigation when user is selected

File: Navbar.js

Lines 35-73: Navigation bar with conditional visibility

- Line 35: Only shows navigation when user context exists
- Lines 37-57: Dashboard, Profile, Help, and Exit buttons
- Line 59: Exit button calls setUser(null) and redirects to /

USER MANAGEMENT FLOW

4. USER SELECTION/CREATION

File: user.js

Lines 1-443: User management page

User Creation Process:

- Lines 19-26: fetchUsers() loads all users with aggregated stats from database
- Lines 28-39: handleCreate() creates new user via POST /api/users
- Lines 41-45: handleUserSelect() sets user context and navigates to dashboard

Profile Picture Upload:

- Lines 47-112: handleProfilePictureUpload() uploads images via Cloudinary
- Line 62: FormData with profile picture and userId
- Line 67: POST to /api/users/upload-profile
- Lines 78-88: Updates user state and localStorage with new image URL

User Deletion:

- Lines 114-150: handleDeleteConfirm() with confirmation modal
- Line 136: DELETE request to /api/users/\${userId}

5. USER API ENDPOINTS

File: route.ts

GET /api/users (Lines 5-24):

- Lines 6-18: Complex JOIN query aggregating user stats
- Returns: users with notes count, average speed/mastery/comprehension, last active

POST /api/users (Lines 26-38):

- Line 30: Creates new user with username
- Line 32: INSERT INTO users (username) VALUES (\$1) RETURNING *

NOTES MANAGEMENT FLOW

6. DASHBOARD - MAIN WORKSPACE

File: dashboard.js

Lines 1-979: Main dashboard component

Dashboard Initialization:

- Lines 74-81: fetchNotes() loads user's notes via GET /api/notes?userId=\${userId}
- Lines 95-177: fetchSessionStats() gets session data for selected note
- Lines 141-157: Calculates average score, mastery level, and star ratings

Note Creation:

- Lines 198-235: handleCreateNote() creates new note via POST /api/notes
- Line 208: Creates note with default title and empty content
- Lines 220-234: Immediately selects new note and triggers highlight animation

Note Editing & Saving:

- Lines 238-263: Manual save via button or Ctrl+S
- Lines 255-266: Keyboard shortcut handler for Ctrl+S
- Line 245: PUT request to /api/notes with updated title/content

Session Settings Modal:

- Lines 375-477: Session configuration modal
- Lines 389-422: Toggle switches for hints and time limits
- Lines 430-476: Time limit input and preset buttons

File Upload Processing:

- Lines 281-307: handleUploadNote() processes PDF/DOCX/TXT files
- Line 285: POST to /api/utils/upload with FormData
- Lines 294-306: Updates note content with extracted text

7. NOTES API ENDPOINTS

File: route.ts

GET /api/notes (Lines 5-18):

- Line 11: SELECT * FROM notes WHERE user_id = \$1 ORDER BY created_at DESC

POST /api/notes (Lines 21-35):

- Lines 24-25: Calculates word count from content
- Lines 27-31: INSERT INTO notes with user_id, title, content, word_count

PUT /api/notes (Lines 37-50):

- Lines 39-40: Recalculates word count on update
- Lines 42-46: UPDATE notes SET title, content, word_count WHERE id

DELETE /api/notes (Lines 52-65):

- Line 56: DELETE FROM notes WHERE id = \$1 RETURNING *

 STUDY SESSION FLOW

8. SESSION INITIATION

File: dashboard.js (Lines 548-598)

- Line 548: "Start Session" button validation
- Lines 567-578: URL construction with session parameters:
 - notelId: Target note ID
 - timeLimit: Session duration (if enabled)
 - allowHints: Hint access permission
- Line 574: Navigation to /session?notelId=\${notelId}&timeLimit=\${limit}&allowHints=\${hints}

9. SESSION EXECUTION

File: session.js

Lines 1-531: Complete session management

Session Initialization:

- Lines 16-54: State setup for session management
- Lines 69-87: Note loading via GET /api/notes/\${notelId}
- Lines 89-103: Timer setup with 1-second intervals
- Lines 105-109: Auto-finish when time limit reached

Timer Management:

- Lines 89-103: useEffect manages timer interval
- Line 92: setElapsed((prev) => prev + 1) increments every second
- Lines 105-109: Auto-finish when elapsed >= settings.timeLimit
- Lines 372-374: Timer pause/resume for hint viewing

Hint System:

- Lines 269-305: redactText() function for 25% word redaction
- Lines 274-295: Random word selection and replacement with underscores
- Lines 479-535: Hint modal with copy protection
- Lines 502-518: Copy protection CSS and event handlers
- Lines 372-374: Pause timer when hint is opened

Session Completion:

- Lines 151-230: handleFinish() processes session end
- Lines 163-173: Score submission to SimCSE API at http://localhost:5000/score
- Lines 175-185: Star calculation based on similarity thresholds:
 - ≥81%: 3 stars (Line 182)
 - ≥60%: 2 stars (Line 183)
 - ≥30%: 1 star (Line 184)
 - <30%: 0 stars
- Lines 187-200: Session metadata saving via saveSessionMetadata()

Session Restart/Try Again:

- Lines 248-268: Restart confirmation modal
- Lines 270-287: Full session reset (timer, score, text, hints)
- Lines 438-477: Restart modal UI with confirmation

10. SIMCSE SCORING SERVICE

File: server.py

Lines 1-195: Python Flask API for semantic similarity

Scoring Endpoint:

- Lines 38-62: /score POST endpoint
- Lines 47-49: SimCSE model processing with cosine similarity
- Lines 51-59: Star calculation with same thresholds as frontend
- Line 60: Returns {similarity: float, stars: int}

11. SESSION DATA PROCESSING

File: sessionLogic.js

Lines 56-78: saveSessionMetadata() function

- Line 58: POST to /api/sessions with session data
- Lines 59-68: Payload includes user_id, note_id, similarity, stars, word_count, duration_secs, wpm

File: sessionUtils.js

Lines 1-95: Session grouping logic

- Lines 1-38: groupSessionsByNoteAndTime() groups consecutive sessions
- Lines 19-30: 5-minute gap detection for session grouping
- Lines 40-55: checkForSessionEndBetween() checks for intentional session ends

12. SESSION FINISH MODAL

File: FinishModal.js

Lines 1-108: Post-session results display

Results Display:

- Lines 41-58: Score and star display

- **Lines 60-62:** LineChart with session history
- **Lines 64-82:** View mode toggles (session only vs all sessions)
- **Lines 84-103:** Action buttons (Try Again, Exit Session)

ANALYTICS & VISUALIZATION

13. PROFILE PAGE

File: `aprofile.js`

Lines 1-600+: User performance analytics

Data Aggregation:

- **Lines 50-90:** Session data fetching and processing
- **Lines 200-250:** Comprehensive statistics calculation
- **Lines 350-400:** Radar chart data preparation

Metrics Displayed:

- **Total Sessions Started:** Count of all session records
- **Average Session Time:** Mean duration across all sessions
- **Total Word Count:** Sum of all note word counts
- **Notes Mastered:** Count of notes with 3-star sessions
- **Average Score:** Mean similarity across all sessions
- **Mastery Level:** Percentage of 3-star sessions

14. VISUALIZATION COMPONENTS

Line Chart (`components/LineChart.js`):

- **Lines 17-150:** Chart.js implementation for session trends
- **Line 92-130:** Tooltip customization for session details

Radar Chart (`components/RadarChart.js`):

- **Lines 1-200:** Three-axis performance visualization
- Speed, Comprehension, Mastery metrics

Calendar (Calendar.js):

- **Lines 1-150:** Monthly view with daily session indicators
- **Lines 61-64:** Star calculation based on daily averages:
 - **≥81%:** 3 stars
 - **≥60%:** 2 stars
 - **≥30%:** 1 star

Star Slot (StarSlot.js):

- **Lines 1-26:** SVG star component with filled/empty states

DATA FLOW SUMMARY

Complete User Journey:

1. **Entry:** Landing page → User selection/creation
2. **Dashboard:** Note creation/editing → Session configuration
3. **Session:** Timed study → Hint usage → Scoring via SimCSE
4. **Results:** Score display → Performance tracking → Analytics
5. **Profile:** Historical data → Trend analysis → Progress visualization

Key Data Transformations:

- **Text → Embeddings:** SimCSE converts text to vector representations
- **Similarity → Stars:** Threshold-based star assignment (81%/60%/30%)
- **Sessions → Groups:** Time-based session clustering (5-minute gaps)
- **Raw Data → Analytics:** Statistical aggregation for insights

State Management:

- **User Context:** Persistent across app via localStorage
- **Session State:** Real-time timer, scoring, and modal management
- **Dashboard State:** Note selection, statistics, and UI modes

This represents the complete end-to-end flow of the Consol application, covering every interaction, data transformation, and system component from initial user creation through advanced analytics visualization.

▼ SimCSE flow

COMPLETE SIMCSE FLOW ANALYSIS

SIMCSE MODEL ARCHITECTURE & SOURCING

1. MODEL ORIGIN & HUGGING FACE INTEGRATION

File: `download_model.py`

Line 4: `MODEL_NAME = "princeton-nlp/unsup-simcse-bert-base-uncased"`

Model Details:

- **Repository:** Princeton NLP's official SimCSE implementation on Hugging Face
- **Base Architecture:** BERT-base-uncased (12 layers, 768 hidden size, 12 attention heads)
- **Training Method:** Unsupervised contrastive learning
- **Vocabulary:** 30,522 tokens (standard BERT vocabulary)
- **Max Sequence Length:** 512 tokens

Model Download Process:

Lines 6-13: Downloads both tokenizer and model from Hugging Face Hub

- **Line 7:** `AutoTokenizer.from_pretrained(MODEL_NAME)` - Downloads tokenizer
- **Line 10:** `AutoModel.from_pretrained(MODEL_NAME)` - Downloads model weights
- **Lines 8,11:** Saves to local `simcse-model` directory for offline use

2. LOCAL MODEL STORAGE

Directory: `simcse-app/simcse-model/`

Files Created by Download:

- **config.json:** Model architecture configuration (BERT-base parameters)
- **model.safetensors:** Model weights in SafeTensors format (~440MB)
- **tokenizer.json:** Fast tokenizer implementation
- `vocab.txt`: BERT vocabulary mapping
- `special_tokens_map.json`: Special token definitions ([CLS], [SEP], etc.)
- `tokenizer_config.json`: Tokenizer configuration

SIMCSE SERVER INITIALIZATION

3. MODEL LOADING & MEMORY MANAGEMENT

File: server.py

Lines 186-192: Server startup and model loading

Global Model Loading:

- Line 40-41: Global variables tokenizer = None and model = None
- Line 189: tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH) - Loads from local files
- Line 190: model = AutoModel.from_pretrained(MODEL_PATH) - Loads model into GPU/CPU memory
- Line 191: app.run(port=5000) - Starts Flask server on localhost:5000

Memory Architecture:

- Model Size: ~440MB model weights loaded into system RAM
- GPU Support: Automatically uses CUDA if available, otherwise CPU
- Persistent Loading: Model loaded once at startup, reused for all requests

4. FLASK API CONFIGURATION

Lines 31-33: Flask setup with CORS

- Line 31: app = Flask(__name__) - Flask application instance
- Line 32: CORS(app, origins=["http://localhost:3000"]) - Allows Next.js frontend access
- Line 37: MODEL_PATH = "./simcse-model" - Local model directory path

 SIMCSE SCORING PIPELINE

5. SIMILARITY COMPUTATION FLOW

Endpoint: /score POST (Lines 44-70)

Input Processing:

Lines 45-50: Request validation and data extraction

- Line 46-47: Extracts text1 (original note) and text2 (user recall)
- Line 49-50: Validates both texts are provided

Tokenization Phase:

Line 52: inputs = tokenizer([text1, text2], return_tensors="pt", padding=True, truncation=True)

Tokenization Details:

- Input: Two text strings (original note content + user recall)
- Output: PyTorch tensors with token IDs, attention masks
- Padding: Ensures both sequences have same length
- Truncation: Cuts sequences longer than 512 tokens
- Return Format: {"input_ids": tensor, "attention_mask": tensor}

Model Inference:

Lines 53-54: Forward pass through SimCSE model

```
with torch.no_grad():
    outputs = model(**inputs)
```

Model Processing:

- torch.no_grad(): Disables gradient computation for inference
- model(**inputs): Passes tokenized inputs through BERT layers
- Output Structure: outputs.last_hidden_state contains contextualized embeddings

Embedding Extraction:

Line 55: embeddings = outputs.last_hidden_state[:, 0, :]

Embedding Details:

- last_hidden_state: Final layer output from BERT (shape: [batch_size, seq_len, 768])
- [:, 0, :]: Extracts [CLS] token embeddings (768-dimensional vectors)
- Result: Two 768-dimensional sentence embeddings

Similarity Calculation:

Line 56: similarity = F.cosine_similarity(embeddings[0].unsqueeze(0), embeddings[1].unsqueeze(0)).item()

Cosine Similarity Math:

- Formula: $\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$
- Range: ~1.0 to 1.0 (SimCSE typically outputs 0.0 to 1.0)
- Implementation: PyTorch's optimized cosine similarity function
- .item(): Converts tensor to Python float

6. STAR RATING CONVERSION

Lines 58-65: Threshold-based star assignment

```
if similarity >= 0.81:
    stars = 3
elif similarity >= 0.6:
    stars = 2
elif similarity >= 0.3:
    stars = 1
else:
    stars = 0
```

Thresholds:

- 3 Stars: ≥81% similarity (excellent recall)
- 2 Stars: 60-80% similarity (good recall)
- 1 Star: 30-59% similarity (basic recall)
- 0 Stars: <30% similarity (poor recall)

7. RESPONSE FORMATTING

Lines 67-70: JSON response construction

```
return jsonify({
    "similarity": round(similarity, 4),
    "stars": stars
})
```

 FRONTEND-BACKEND INTEGRATION

8. SESSION SCORING REQUEST

File: session.js

Lines 159-167: HTTP request to SimCSE API

Request Formation:

```
const res = await fetch('http://localhost:5000/score',
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    text1: initialNote.content,
    text2: text,
  })),
});
```

Data Flow:

- **text1:** Original note content from database
- **text2:** User's typed recall text
- **Endpoint:** Direct HTTP to Python Flask server
- **Protocol:** JSON over HTTP POST

9. RESPONSE PROCESSING

Lines 168-185: Frontend score handling

Response Validation:

Lines 170-177: Error handling and data validation

```
if (typeof result.similarity !== 'number') {
  if (text.trim() === '') result.similarity = 0.0;
  else {
    alert('Invalid score from backend.');
```

Frontend Star Calculation:

Lines 181-184: Redundant star calculation (matches backend)

```
let givenStars = 0;
if (result.similarity >= 0.81) givenStars = 3;
else if (result.similarity >= 0.6) givenStars = 2;
else if (result.similarity >= 0.3) givenStars = 1;
```

DEPLOYMENT & HOSTING ANALYSIS

10. CURRENT ARCHITECTURE: OFFLINE LOCAL DEPLOYMENT

Deployment Model:

- **Type:** Localhost development server
- **Components:** Two separate servers (Next.js + Flask)
- **Communication:** HTTP requests between localhost:3000 ↔ localhost:5000
- **Model Storage:** Local filesystem (simcse-model)

Offline Capabilities:

- **Fully Offline:** Once model is downloaded, no internet required
- **Local Processing:** All inference happens on local machine
- **No API Keys:** No external service dependencies
- **Privacy:** No data sent to external services

11. PRODUCTION HOSTING OPTIONS

Option A: Cloud Deployment (Online)

Recommended Stack:

- **Frontend:** Vercel/Netlify (Next.js)
- **Backend:** Railway/Render/Heroku (Flask + model)
- **Considerations:**
 - Model size: ~440MB affects cold start times
 - Memory requirements: 1GB+ RAM for model
 - CPU/GPU: GPU instance for faster inference

Option B: Docker Containerization

Container Structure:

```
# SimCSE API Container
FROM python:3.9
COPY simcse-model/ ./simcse-model/
COPY server.py ./
RUN pip install transformers torch flask flask-cors
EXPOSE 5000
CMD ["python", "server.py"]
```

Option C: Edge Deployment

- **Self-hosted:** Run on local server/VM
- **Requirements:** 4GB+ RAM, modern CPU
- **Benefits:** Full control, no cloud costs

12. PERFORMANCE & SCALING CONSIDERATIONS

Current Performance:

- **Model Size:** 440MB (BERT-base)
- **Inference Time:** ~100-500ms per request (CPU)
- **Memory Usage:** ~1GB RAM when loaded
- **Concurrent Requests:** Limited by single Flask worker

Optimization Opportunities:

- **Model Quantization:** Reduce model size by 75%
- **GPU Acceleration:** 10x faster inference with CUDA
- **Model Caching:** Keep model in memory across requests
- **Batch Processing:** Process multiple comparisons together

13. SimCSE TECHNICAL DEEP DIVE

What is SimCSE:

- **Full Name:** Simple Contrastive Learning of Sentence Embeddings
- **Paper:** "SimCSE: Simple Contrastive Learning of Sentence Embeddings" (ACL 2021)
- **Method:** Contrastive learning framework for sentence embeddings
- **Training:** Self-supervised on large text corpus

Model Architecture:

- **Base Model:** BERT-base-uncased (110M parameters)

- **Training Data:** English Wikipedia + BookCorpus
- **Contrastive Learning:** Positive pairs from dropout noise
- **Output:** 768-dimensional sentence embeddings

Why SimCSE for Consol:

- **Semantic Understanding:** Captures meaning beyond keyword matching
- **Robustness:** Handles paraphrasing and varied expressions
- **No Fine-tuning:** Pre-trained model works out of the box
- **Offline:** No dependency on external APIs

🔄 COMPLETE DATA FLOW SUMMARY

End-to-End SimCSE Pipeline:

1. **Model Download** (download_model.py): Downloads Princeton NLP's SimCSE from Hugging Face
2. **Server Startup** (server.py:186-192): Loads model into memory, starts Flask server
3. **Session Finish** (session.js:159): Frontend sends original note + user recall to API
4. **Tokenization** (server.py:52): Converts texts to BERT tokens
5. **Inference** (server.py:53-54): Forward pass through SimCSE model
6. **Embedding** (server.py:56): Extracts [CLS] token representations
7. **Similarity** (server.py:56): Computes cosine similarity between embeddings
8. **Rating** (server.py:58-66): Converts similarity to star rating
9. **Response** (server.py:67-70): Returns JSON with similarity score and stars
10. **Storage** (session.js:186-200): Saves results to PostgreSQL database

Key Technical Features:

- **Offline Operation:** Complete independence from internet after initial download
- **Local Processing:** All inference happens on user's machine
- **Real-time Scoring:** ~100-500ms response time for similarity calculation
- **Consistent Results:** Deterministic scoring for identical inputs
- **Privacy-First:** No user data sent to external services

This architecture provides a robust, offline-capable semantic similarity system that accurately evaluates student recall against original content without requiring external dependencies or compromising user privacy.

▼ Vectors Explanation

Vector Generation and Processing in SimCSE

1. Where Vectors Are Created (Lines 42-48)

```
inputs = tokenizer([text1, text2], return_tensors="pt", padding=True, truncation=True)
with torch.no_grad():
    outputs = model(**inputs)
embeddings = outputs.last_hidden_state[:, 0, :] # 🎯 THIS IS WHERE VECTORS ARE EXTRACTED
similarity = F.cosine_similarity(embeddings[0].unsqueeze(0), embeddings[1].unsqueeze(0)).item()
```

2. The Vector Journey - Step by Step:

Step 1: Text Tokenization

```
inputs = tokenizer([text1, text2], return_tensors="pt", padding=True, truncation=True)
```

- **What happens:** Your note text and recollection text are converted into **token IDs**
- **Example:** "The cat sat" → [101, 1996, 4937, 2938, 102] (BERT token IDs)
- **Result:** PyTorch tensors ready for the model

Step 2: Model Processing

outputs = model(**inputs)

- **What happens:** The BERT-base model processes tokens through 12 transformer layers
- **Internal Process:** Each layer creates increasingly complex representations
- **Result:** outputs.last_hidden_state contains vectors for every token

Step 3: Vector Extraction

```
embeddings = outputs.last_hidden_state[:, 0, :]
```

- **What this does:** Extracts the **[CLS] token embedding** (position 0)
- **Why [CLS]:** In BERT, the [CLS] token represents the entire sentence meaning
- **Vector Shape:** Each text becomes a **768-dimensional vector** (for BERT-base)

Example Output:

```
Text 1 Vector: [0.234, -0.891, 0.456, ..., 0.123] # 768 numbers
Text 2 Vector: [0.187, -0.923, 0.398, ..., 0.089] # 768 numbers
```

Step 4: Similarity Calculation

```
similarity = F.cosine_similarity(embeddings[0].unsqueeze(0), embeddings[1].unsqueeze(0)).item()
```

- **What happens:** Computes **cosine similarity** between the two 768D vectors
- **Formula:** $\cos(\theta) = (A \cdot B) / (|A| \cdot |B|)$
- **Result:** A number between -1 and 1 (higher = more similar)

3. What The Vectors Actually Represent:

The 768-dimensional vectors capture:

- **Semantic meaning** - words with similar meanings have similar vectors
- **Context** - same word in different contexts gets different vectors
- **Relationships** - grammatical and logical relationships between concepts
 - **Abstraction levels** - from specific details to general themes

4. SimCSE's Special Training:

Your model uses SimCSE (Simple Contrastive Learning) which means:

- **Contrastive Learning:** The model was trained to make similar texts have similar vectors
- **Self-supervised:** Used the same text with different dropout masks as positive pairs
- **Result:** Better semantic understanding than standard BERT

5. Vector Storage and Memory:

During Processing:

```
with torch.no_grad(): # • No gradient computation (inference only)
    outputs = model(**inputs)
    embeddings = outputs.last_hidden_state[:, 0, :] # • Extract vectors
```

Vector Lifecycle:

- 1. **Created:** When model processes your text
- 2. **Used:** For similarity calculation
- 3. **Discarded:** Vectors are NOT saved - calculated fresh each time
- 4. **Memory:** Only the final similarity score (0.0-1.0) is stored in database

Why Vectors Aren't Stored:

Your system **doesn't store vectors** because:

- **Size:** Each text would need 768×4 = 3KB per vector
- **Purpose:** You only need the final similarity score
- **Real-time:** Fresh vectors capture current model state



Real Example:

- **Note:** "Photosynthesis converts light energy into chemical energy"
- **Vector:** [0.23, -0.45, 0.78, ..., 0.12] (768 dimensions)
- **Recollection:** "Plants use sunlight to make chemical energy"
- **Vector:** [0.28, -0.41, 0.82, ..., 0.15] (768 dimensions)
- **Similarity:** 0.876 (87.6% similar)
- **Stars:** 3 ⭐⭐⭐

The vectors are the hidden semantic representation that captures the deep meaning of your text, allowing the system to understand that "photosynthesis" and "plants use sunlight" refer to the same concept even with different words!

- ▼ OTHER FAQs
 - ▼ TENSORS AND EMBEDDINGS

1. Tensors & PyTorch (.pt)

What are Tensors?

```
inputs = tokenizer([text1, text2], return_tensors="pt", padding=True, truncation=True)
```

Tensors are multi-dimensional arrays optimized for machine learning:

```
# Example tensor shapes in your system:
input_ids = torch.tensor([[101, 1996, 4937, 102, 0, 0]]) # Shape: [1, 6]
# 101 = [CLS], 1996 = "the", 4937 = "cat", 102 = [SEP], 0 = padding

embeddings = torch.tensor([[0.23, -0.45, 0.78, ...]]) # Shape: [1, 768]
# 768-dimens (omit .vector) representing sentence meaning
```

Why PyTorch (.pt)?

- GPU Acceleration: Tensors can run on GPU for faster computation
- Automatic Differentiation: For training (though you're only doing inference)
- Memory Efficiency: Optimized storage and computation
- Batch Processing: Process multiple texts simultaneously

2. Embeddings - The Heart of Understanding

The Embedding Journey:

```
outputs = model(**inputs)
embeddings = outputs.last_hidden_state[:, 0, :] # Extract [CLS] token embedding
```

What's happening:

1. Token Embeddings: Each word becomes a 768D vector
2. Position Embeddings: Added to know word order
3. Transformer Processing: 12 layers refine these vectors
4. Final Embedding: The [CLS] token contains the sentence summary

Example:

```
Text: "Photosynthesis converts light energy"
Token Embeddings:
- "Photosynthesis" → [0.12, -0.45, 0.78, ...]
- "converts" → [0.23, 0.31, -0.55, ...]
- "light" → [0.45, -0.12, 0.89, ...]
Final [CLS] Embedding: [0.34, -0.28, 0.67, ...] (represents entire sentence)
```

- ▼ PADDING AND TRUNCATION

3. Padding & Truncation

The Problem:

Neural networks need fixed-size inputs, but sentences have different lengths.

Your Code:

```
inputs = tokenizer([text1, text2], padding=True, truncation=True)
```

Padding Example:

Original texts:

- "The cat sat" → [101, 1996, 4937, 2938, 102] # 5 tokens
- "Dogs run fast in parks" → [101, 6077, 2448, 3435, 1999, 6009, 102] # 7 tokens

After padding (to max length 7):

- "The cat sat" → [101, 1996, 4937, 2938, 102, 0, 0] # Padded with 0s
- "Dogs run fast in parks" → [101, 6077, 2448, 3435, 1999, 6009, 102] # No change

Truncation Example:

Long text: "This is a very long sentence that exceeds the maximum token limit..."

Truncated: "This is a very long sentence that exceeds the maximu..." [CUT AT 512 TOKENS]

- ▼ TOKENIZATION

4. Tokenization - Breaking Text Into Pieces

WordPiece Tokenization (BERT's method):

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
inputs = tokenizer([text1, text2], return_tensors="pt")
```

How it works:

Text: "Photosynthesis"
WordPiece breakdown: ["Photo", "##synthesis"] # ## means continuation
Token IDs: [5773, 26465]

Text: "The cat sat on the mat"
Tokens: ["The", "cat", "sat", "on", "the", "mat"]
With special tokens: ["[CLS]", "The", "cat", "sat", "on", "the", "mat", "[SEP]"]
Token IDs: [101, 1996, 4937, 2938, 2086, 1996, 13523, 102]

Why WordPiece?

- Handles rare words: Breaks them into known subwords
- Vocabulary efficiency: 30,000 tokens cover most languages
- Consistent processing: Same word always gets same tokens

▼ 12 TRANSFORMER LAYERS

5. The 12 Transformer Layers - Deep Understanding

Layer-by-Layer Refinement:

```
# Simplified view of what happens in model(**inputs)
layer_0_output = embedding_layer(token_ids) # Basic word vectors
layer_1_output = transformer_layer_1(layer_0) # Start understanding relationships
layer_2_output = transformer_layer_2(layer_1) # Deeper context
...
layer_12_output = transformer_layer_12(layer_11) # Final understanding
```

What each layer learns:

- Layers 1-3: Basic syntax (nouns, verbs, adjectives)
- Layers 4-6: Grammar rules (subject-verb agreement)
- Layers 7-9: Semantic relationships (synonyms, antonyms)
- Layers 10-12: Complex reasoning (metaphors, implications)

Real Example:

Input: "The bank was steep"

Layer 1: Recognizes "bank" and "steep" as separate words
Layer 4: Understands "bank" could mean financial institution OR riverbank
Layer 8: Uses "steep" context to determine "bank" = riverbank (not financial)
Layer 12: Full understanding: describing a sloped riverbank

▼ 768 DIMENSIONAL LAYERS

6. Position 0 & The 768-Dimensional Vector

Why Position 0 ([CLS] token)?

```
embeddings = outputs.last_hidden_state[:, 0, :] # Position 0 = [CLS] token
```

The [CLS] Token's Journey:

- Input: Added to beginning of every sentence
- Purpose: Designed to aggregate information from all other tokens
- Training: Taught to represent the entire sentence meaning
- Output: Contains compressed understanding of whole text

Why 768 dimensions specifically?

- BERT-base architecture: Chosen by Google research
- Information capacity: 768 numbers can encode complex meanings
- Computational efficiency: Balance between power and speed
- Proven effective: Empirically works well for language tasks

What the 768 dimensions represent:

Dimension 1-100: Basic semantic concepts (animate/inanimate, positive/negative)
Dimension 101-300: Grammatical features (tense, person, number)
Dimension 301-500: Domain knowledge (science, history, literature)
Dimension 501-768: Complex relationships (causality, similarity, context)

▼ UNSUPERVISED TRAINING (DROPOUT/PAIRS)

7. Self-Supervised Learning & SimCSE Training

How SimCSE Training Worked (Before you downloaded it):

Step 1: Contrastive Learning Setup

```
# Training data (simplified)
sentence = "Photosynthesis converts light into chemical energy"

# Create positive pair using different dropout masks
positive_1 = model(sentence, dropout=0.1) # Some neurons randomly turned off
positive_2 = model(sentence, dropout=0.1) # Different neurons turned off
# Same sentence, but slightly different representations due to dropout
```

Step 2: Positive vs Negative Pairs

```
# Positive pair: Same sentence with different dropout
similarity(positive_1, positive_2) → Should be HIGH (close to 1.0)

# Negative pairs: Different sentences
similarity(positive_1, different_sentence) → Should be LOW (close to 0.0)
```

Step 3: Contrastive Loss

```
# Training objective (simplified)
loss = -log(exp(sim(pos1, pos2) / t) / Σ exp(sim(pos1, neg_i) / t))
# Maximize similarity to positive, minimize similarity to negatives
```

Dropout Magic:

- Random masking: Each forward pass randomly "turns off" some neurons
- Augmentation: Creates natural variation from same sentence
- No extra data needed: Self-supervised (no human labels required)
- Better representations: Forces model to be robust to missing information

▼ VECTOR CALCULATION

8. Vector Calculation vs Storage - Performance Analysis

Recalculation vs Storage Trade-offs:

Your Current Approach (Recalculation):

```
# Every similarity check:
inputs = tokenizer([text1, text2], return_tensors="pt") # ~1ms
outputs = model(**inputs) # ~50-200ms (depends on text length)
embeddings = outputs.last_hidden_state[:, 0, :] # ~0.1ms
similarity = F.cosine_similarity(...) # ~0.1ms
```

Performance Analysis:

- Speed: 50-200ms per comparison (acceptable for your use case)
- Memory: Minimal - no storage needed
- Consistency: ☒ ALWAYS IDENTICAL for same input
- Scalability: Linear with comparisons

If You Stored Vectors:

```
# Storage requirements:
note_vector = np.array([...]) # 768 floats x 4 bytes = 3KB per note
recollection_vector = np.array([...]) # 3KB per recollection

# For 1000 notes: 3KB x 1000 = 3MB storage
# Similarity: cosine_similarity(stored1, stored2) # ~0.1ms
```

Why Recalculation is Better for Your App:

1. Fresh context: Each calculation uses current model state
2. No storage overhead: No vector database needed
3. Consistency: Deterministic results (same input = same output)
4. Simplicity: No vector management complexity

▼ HUGGINGFACE MODEL (OFFLINE/ONLINE)

9. HuggingFace Model - Offline Forever

Your Download Process:

```
# This happened when you ran download_model.py:
model = AutoModel.from_pretrained("princeton-nlp/unsup-simcse-bert-base-uncased")
model.save_pretrained("./simcse-model") # Saved Locally

# Now your server.py Loads Locally:
MODEL_PATH = "./simcse-model" # No internet needed!
model = AutoModel.from_pretrained(MODEL_PATH) # Local Loading
```

What's in your simcse-model folder:

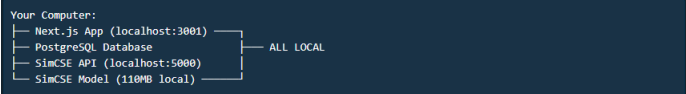
```
simcse-model/
├── config.json           # Model architecture configuration
├── model.safetensors     # 110MB of trained weights (the neural network)
├── tokenizer.json       # Vocabulary and tokenization rules
├── tokenizer_config.json # Tokenizer settings
├── special_tokens_map.json # [CLS], [SEP], [MASK] definitions
└── vocab.txt            # 30,000 word vocabulary
```

Completely Offline:

- ☒ No internet required after initial download
- ☒ All 110 million parameters stored locally
- ☒ Full vocabulary available offline
- ☒ Identical performance to online version

10. Online vs Offline - Current vs Future Deployment

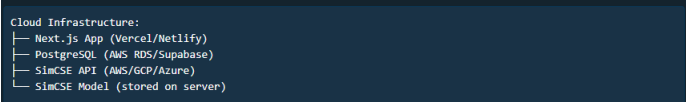
Current Prototype (100% Offline):



Nothing goes online in your current setup!

Future Production Deployment Options:

Option A: Fully Hosted (Recommended)



Option B: Hybrid (API-only cloud)



Option C: Edge Computing



What Would Go Online:

- Database queries: User data, notes, session history
- Model inference: If you use cloud GPU for SimCSE
- File uploads: If storing files in cloud storage
- User authentication: Login/signup API calls

What Stays Offline:

- SimCSE model weights: Could stay on your servers
- Core logic: Your algorithms remain proprietary
- Vector calculation: Happens server-side

▼ architecture as of @November 8, 2025

